Hash Tables: Collision Handling and Performance

A Research-Based Data Structures Project on Linear Probing and Separate Chaining

CMPT 225 Final Project
Jun Travis Ziea

Student #: 301619815

Date: April 16, 2026

Abstract

This project studies an often seen component in data structures: hash-table collision handling. The report focuses on linear probing and separate chaining, and compares them through both theory and implementation. The main goal is to explain how collisions affect performance as the load factor increases and to show why the choice of collision strategy matters in real systems such as databases, caches, compilers, and language runtimes.

The implementation portion includes two C++ hash tables that store integer keys: one using open addressing with linear probing and one using separate chaining with linked lists. The benchmark program measures insertion time, search time, number of collisions, and average probe counts at several load factors. The experimental results included in this report show a clear trade-off: linear probing is extremely fast at moderate load factors because of strong cache locality, but its probe lengths and search cost rise sharply near high occupancy. Separate chaining is less sensitive to high load factors and keeps probe growth modest, though it pays extra overhead from pointer-heavy memory access.

The broader literature shows that modern high-performance hash tables often extend and build upon these ideas. Recent designs such as Swiss tables and Meta's F14 emphasize metadata, SIMD-friendly scanning, cache-aware probing, and careful control of load factor. This makes collision handling a useful topic for connecting textbook data structures to current software engineering practice.

1. Introduction and Motivation

Hash tables are commonly used data structures in computer science because they support expected constant-time insertion, lookup, and deletion [1]. In practice, however, this performance depends heavily on how collisions are handled. A collision occurs when two keys map to the same location. The way a table resolves this conflict determines not only its asymptotic behavior, but also its memory usage, cache behavior, and speed under real workloads [2].

Hash tables appear in systems code everywhere: symbol tables in compilers, page tables and caches in operating systems, indexing in databases, routing and lookup tables in networking software, and

key-value maps in programming languages [1]. Even small implementation improvements can have broad impact at scale.

The focus of the project is how do linear probing and separate chaining behave as the load factor increases? That question is simple enough to implement clearly, but hash table designs are not just a static design where the same type is used every time, hash table designs are still discussed due to the many different ways a table can be designed [3].

2. Background and Definitions

A hash table stores keys in an array of buckets. A hash function maps each key to a bucket index. The load factor is usually defined as $\alpha = \frac{n}{m}$, where n is the number of stored keys and m is the number of buckets. As α grows, collisions become more likely [1].

In separate chaining, each bucket stores a small secondary container, often a linked list, containing all keys that hash to that bucket [2]. In linear probing, all keys stay directly in the main array. If a key's home bucket is occupied, the algorithm probes the next slot, then the next, and so on until an empty slot is found [4].

The main metrics used in this project are: collision count, insertion time, search time, average insertion probes, and average search probes. A probe means one bucket or slot inspection during an operation. These metrics make the performance impact of collisions visible.

3. Core Techniques Compared

Technique	Main Idea	Strengths	Weaknesses
Linear Probing	Store all items in one array; on collision, scan forward to the next free slot.	Very simple; low memory overhead; strong cache locality; often excellent at moderate load factors.	Primary clustering; performance degrades quickly near high load factors; sensitive to hash quality.
Separate Chaining	Each bucket stores a chain/list of colliding keys.	Easy deletion; can tolerate α > 1; performance degrades more smoothly under heavy load.	Extra pointer/storage overhead; weaker cache locality; dynamic allocation costs.

4. Historical Narrative

Hashing has deep roots in early computer science. Linear probing dates back to the 1950s and was already being analyzed by the early 1960s. Along with a formal analysis by Knuth in *The Art of Computer Programming*. That work also explains why linear probing is important on modern machines: sequential memory access interacts well with caches.

Separate chaining developed as the other answer to collisions. It is conceptually simple and robust because colliding keys do not compete for the same contiguous cluster of array slots [1]. These two methods have formed the baseline comparison in textbooks and software libraries.

Research then expanded beyond the basics. Robin Hood hashing, introduced by Celis in the 1980s, rebalances probe distances so that keys with larger displacement can “steal” positions from keys with smaller displacement [5]. Cuckoo hashing, introduced by Pagh and Rodler in 2001, showed that using multiple candidate locations can give worst-case constant lookup time [6]. These developments helped frame collision handling as a design space, not a single technique.

5. Recent Advancements and Real-World Relevance

Modern high-performance hash tables still build on the same basic concerns: collision resolution, locality, and load-factor control. Abseil’s Swiss tables are designed as replacements for standard unordered containers and use a cache-friendly layout plus control-byte metadata to speed lookups and insertions [7]. The documentation states that it maintains a maximum load factor of 7/8 (87.5%) , after which the table grows. This is a useful real-world example of how performance and load factor are tightly linked.

Meta’s F14 family pushes the same idea further by using 14-way probing and filtering techniques to reduce unnecessary comparisons [3]. Meta describes F14 as a good general-purpose default because it balances CPU cost, memory usage, and reference-stability trade-offs. This is evidence that practical performance depends on engineering choices, not just big-O notation.

Recent theory also continues to reference the basics. “Linear Probing Revisited: Tombstones Mark the Death of Primary Clustering” has shown that careful handling of tombstones can counteract primary clustering and propose graveyard hashing, a variant of linear probing that preserves locality while improving behavior under updates [8]. This is especially useful in a report like this one because it shows a clear through-line from textbook linear probing to current research.

6. Implementation

The included code folder contains a complete C++ benchmarking project. The file hash_tables.h implements two classes: LinearProbingHashTable and SeparateChainingHashTable. Both store integer keys and expose insert, contains, size, capacity, load factor, and metric-reporting behavior. The benchmark driver in main.cpp generates random unique keys, fills each table to a target load factor, runs successful and unsuccessful searches, and writes results to benchmark_results.csv.

The implementation was intentionally kept simple to make the comparison easy to understand and under time constraint. Linear probing uses a fixed-size array of slots and wraps around modulo of the

table size [1]. Separate chaining uses a vector of linked lists [2]. For both structures, collisions and probes are counted explicitly so that runtime measurements can be paired with structural explanations.

A useful extension for future work would be to add Robin Hood hashing, quadratic probing, dynamic resizing, deletion-heavy workloads, and larger key types such as strings. That would connect the implementation even more directly to modern literature and practical systems.

7. Experiments and Evaluation

Methodology. The benchmark uses a table capacity of 200,003 buckets and tests target load factors 0.50, 0.70, 0.80, 0.90, and 0.95. For each load factor, the program inserts random unique integers and then performs a mixed search workload containing both present and absent keys. The recorded metrics are insertion time, search time, collisions, and average probe counts.

Important note. The benchmarking program was run through the SFU VDI desktop lab. Results may differ on different machines.

Method	Load	Collisions	Insert (ms)	Search (ms)	Avg Insert Probes	Avg Search Probes
Linear Probing	0.50	49764	5.4237	9.5335	1.4976	1.8305
Separate Chaining	0.50	21447	15.9241	13.2929	1.2515	1.3335
Linear Probing	0.70	161885	8.7911	17.2654	2.156	3.446
Separate Chaining	0.70	39129	23.8767	19.9486	1.346	1.465
Linear Probing	0.80	326244	11.0020	27.8809	3.039	6.518
Separate Chaining	0.80	49609	23.4704	22.8653	1.396	1.531
Linear Probing	0.90	831391	25.9347	82.7472	5.619	20.595
Separate Chaining	0.90	61220	27.5724	27.8449	1.450	1.600
Linear Probing	0.95	1882163	27.7638	257.1135	10.906	79.335
Separate Chaining	0.95	67172	29.3938	28.3749	1.472	1.632

8. Discussion

The experiments above have shown a trade-off. At moderate load factors, linear probing is extremely competitive because it stores all data in one contiguous array. This usually means fewer allocations and

better cache behavior. In the included benchmark, linear probing also inserts faster than separate chaining across all tested load factors.

However, the collision and probe data reveal what happens when overfilling an open-addressed table. Average search probes in linear probing increase sharply from about 1.83 at load factor 0.50 to over 79 at load factor 0.95 in this benchmark. That rise matches the literature on clustering, when long runs of occupied cells form, both hits and misses become expensive [9].

Separate chaining behaves differently. It does not avoid collisions, but it contains them inside bucket-local chains. As a result, the probe growth is much smoother as the table becomes fuller. This makes chaining more stable at very high load factors, although it often loses some constant-factor speed due to poorer locality and pointer chasing [1].

The most important note is that “O(1) expected time” hides differences. Load factor, memory layout, and collision policy all matter [1]. Which is why modern designs such as Swiss tables and F14 have a focus on metadata layout, probing strategy, and practical threshold tuning [3], [7].

9. Conclusion and Future Work

This project looks into collision handling to connect theory, implementation, and modern systems design. Linear probing and separate chaining both achieve expected constant-time dictionary operations, but they do so with different practical strengths [1]. Linear probing is simpler and often faster when the table is kept at a moderate load factor [9]. Separate chaining is more tolerant of heavy occupancy and updates, though it can pay extra memory and locality costs [4].

A strong next step would be to broaden the experimental section by adding dynamic resizing, deletion-heavy workloads, string keys, and one or two modern alternatives such as Robin Hood hashing or a Swiss-table-style design. That would turn this project from a solid baseline study into a deeper exploration of how modern hash tables evolve from classical ideas into high-performance systems [3], [7].

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*. MIT Press, 2009.

[2] GeeksforGeeks, “Collision Resolution Techniques,” *GeeksforGeeks*, Jan. 18, 2023. https://www.geeksforgeeks.org/dsa/collision-resolution-techniques/ (accessed Apr. 16, 2026).

[3] N. Bronson, “Open-sourcing F14 for faster, more memory-efficient hash tables,” *Engineering at Meta*, Apr. 25, 2019. https://engineering.fb.com/2019/04/25/developer-tools/f14/ (accessed Apr. 16, 2026).

[4] GeeksforGeeks, “Open Addressing Collision Handling technique in Hashing,” *GeeksforGeeks*, Aug. 12, 2015. https://www.geeksforgeeks.org/dsa/open-addressing-collision-handling-technique-in-hashing/ (accessed Apr. 16, 2026).

[5] P. Celis, P.-A. Larson, and J. Ian Munro, “Robin hood hashing,” Jan. 1985, doi: https://doi.org/10.1109/sfcs.1985.48.

[6] R. Pagh and F. F. Rodler, “Cuckoo hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, May 2004, doi: https://doi.org/10.1016/j.jalgor.2003.12.002.

[7] “abseil / Swiss Tables Design Notes,” *Abseil.io*, 2017. https://abseil.io/about/design/swisstables

[8] M. A. Bender, B. C. Kuszmaul, and W. Kuszmaul, “Linear Probing Revisited: Tombstones Mark the Death of Primary Clustering,” *arXiv.org*, 2021. https://arxiv.org/abs/2107.01250 (accessed Apr. 17, 2026).

[9] A. Pagh, R. Pagh, and M. Ružić, “Linear Probing with Constant Independence,” *SIAM Journal on Computing*, vol. 39, no. 3, pp. 1107–1120, Jan. 2009, doi: https://doi.org/10.1137/070702278.